Modulo2-JF

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

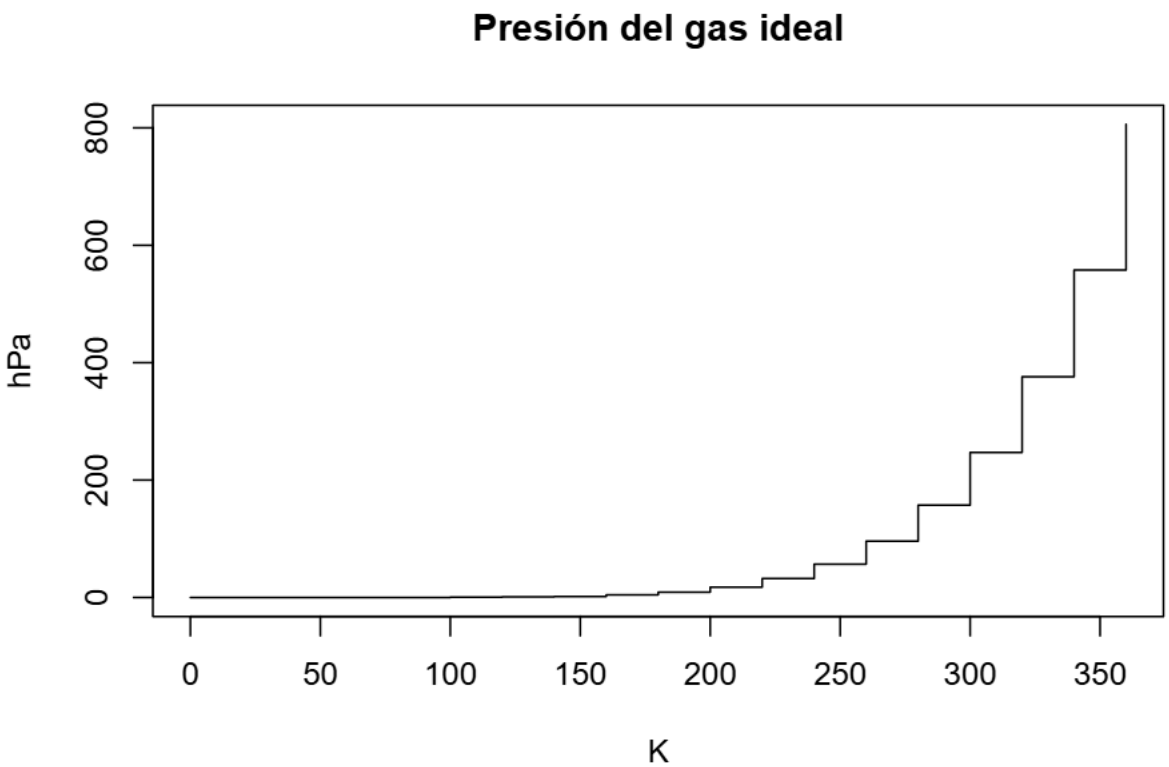
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 16.678831 13.936077 18.969711 17.209225 18.892568 18.232602 21.077887
##   [8] 20.501637 18.503388 29.200316 15.816490 23.090894 27.627107 19.627487
##  [15] 26.146436 22.467147 24.755276 22.067155 16.540685 28.658943 25.082107
##  [22] 20.812074 24.483443 27.963940 31.803238 18.809486 16.319920 19.649756
##  [29] 25.193278 28.729494 27.355818 17.780372 31.183423 18.833011 23.673239
##  [36] 20.124786 15.632797 24.198471 28.204684 25.180072 15.288221 16.681768
##  [43] 22.694072 23.612150 16.955707 20.864501 26.643298 22.660389 29.863942
##  [50]  9.749687 11.330354 25.689898 11.506083 19.856649 18.171981 21.801421
```

```
##  [57] 28.109510 33.706576 17.787074 27.229065 15.988294 27.155172 17.956245
##  [64] 17.121523 28.287288 21.340290 28.780949 29.445845 20.767403 19.890172
##  [71] 24.267753 24.495048 29.867452 28.155262 20.673972 17.851065 30.650852
##  [78] 23.194980 21.372064 30.952254 21.611887 17.432008 25.369258 25.740571
##  [85] 21.195977 28.445811 22.620479 24.733163 22.916525 22.506088 12.411545
##  [92] 14.858173 22.518467 16.729404 30.972487 20.676077 23.290746 24.835634
##  [99] 28.245779 34.552853 21.905588 23.619766 19.768159 29.844388 19.495591
## [106] 13.824505 26.160888 31.273116 32.950945 22.161205 23.844173 10.409163
## [113] 22.435681 15.453109 30.072167 11.915101 23.205684 19.567284 22.016096
## [120] 11.202650 25.560336 22.649290 21.265660 26.088334 17.947156 28.166425
## [127] 26.740240 24.269336 21.472574 18.070155 21.901744 20.949117 23.072161
## [134] 23.191697 24.380613 23.370950 21.901006 20.211943 27.276139 24.597632
## [141] 19.287628 20.482334 19.574993 20.105506 20.713319  4.869721 29.535420
## [148] 25.602861 17.786401 23.846606 21.746340 23.447269 28.096937 27.420153
## [155] 15.124303 33.924262 17.506021 19.690952 20.462022 24.305123 22.375192
## [162] 14.130844  9.841097 14.407152 23.550856 27.480228 24.848247 21.252652
## [169] 19.184762 20.014685 18.241884 19.213210 17.992748 21.158537 27.530320
## [176] 23.063417 16.471510 17.840759 27.214874 17.519761 15.221476 23.506881
## [183] 22.879104 18.632034 22.777343 21.913542 35.338892 21.034464 23.283287
## [190] 17.752309 26.795642 20.328294 25.028157 20.635190 30.388013 13.661011
## [197] 22.950127 31.360989 29.862241 26.158226 14.609414 17.989883 18.707989
## [204] 24.036910 26.545100 16.356063 28.108433 20.494976 21.730078 23.051611
## [211] 22.956996 23.105624 21.036404 19.270794 15.936025 20.770386 25.178530
## [218] 28.160350 21.386609 28.281680 13.770652 18.184262 22.633984 25.390069
## [225] 21.460528 23.970024 19.193074 25.198018 20.561959 19.148426 30.418645
## [232] 24.282359 21.078448 21.503454 24.174260 24.785245 17.353353 21.869707
## [239] 19.261369  7.554836 20.437347 17.996805 26.819912 20.208049 16.484267
## [246] 24.552223 25.473259 23.004731 11.549977 24.253583 26.736672 18.545357
## [253] 17.398499 20.021847 26.110381 26.123222 21.734751 32.322633 19.124062
## [260] 20.112238 36.679415 21.491624 25.997689 26.336258 27.710232 19.302163
## [267] 24.367250 23.590921 23.222836 27.282239 20.948732 24.015754 20.159305
## [274] 19.893581 33.754767 23.532538 24.495248 23.962219 14.705821 29.168962
## [281] 25.101068 26.863599 24.769323 23.979112 20.093063 15.953051 16.142216
## [288] 26.454455 25.421449 17.673219 16.393256 20.352618 18.410595 17.205641
## [295] 29.749041 23.741931 21.852648 23.588704 15.449453 25.009683 25.463624
## [302] 30.457962 15.579425 28.607124 24.929052 30.111956 25.850439 25.092470
## [309] 24.173126 18.242559 15.432945 10.836514 25.783458 25.058392 23.472588
## [316] 29.844972 24.074537 18.954008 22.573472 26.060223 19.796415 23.002785
## [323] 17.610670 18.855649 26.567937 22.851517 21.273757 20.077140 21.579910
## [330] 15.759629 23.657261 22.810865 19.806802 20.635460 12.719838 14.374880
## [337] 15.988977 25.958492 18.574563 30.578818 20.989358 17.859657 19.796450
## [344] 32.819723 18.704996 16.048340 25.784013 21.551198 21.709503 23.012941
```

```r
w1 <- length(z1)
w1
```

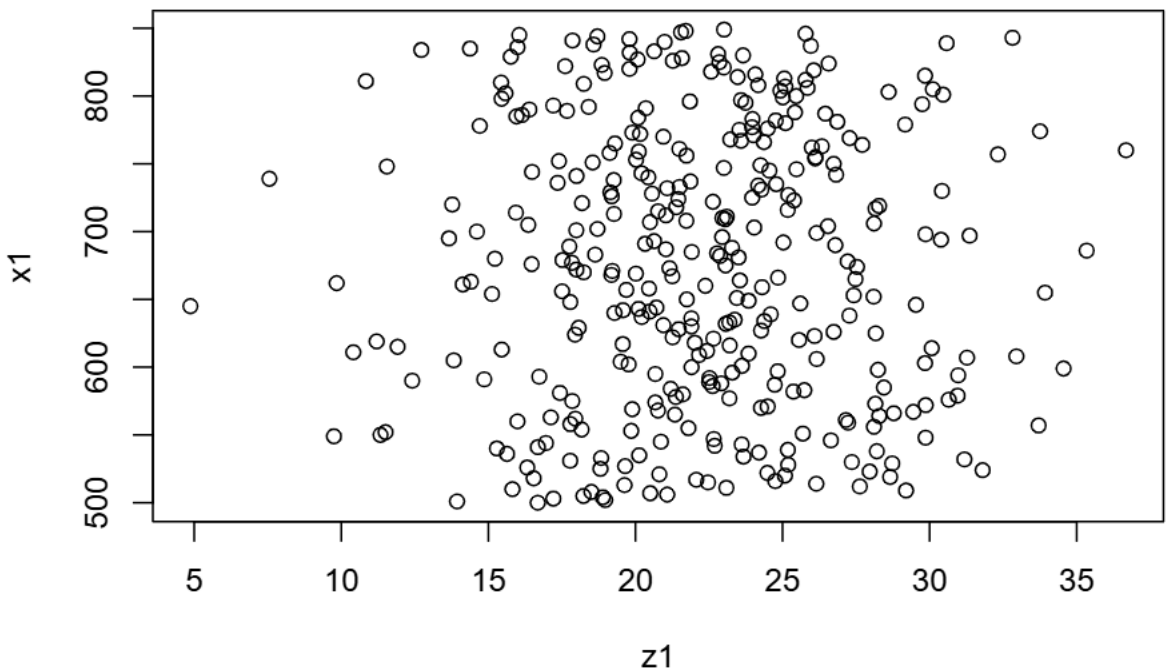
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

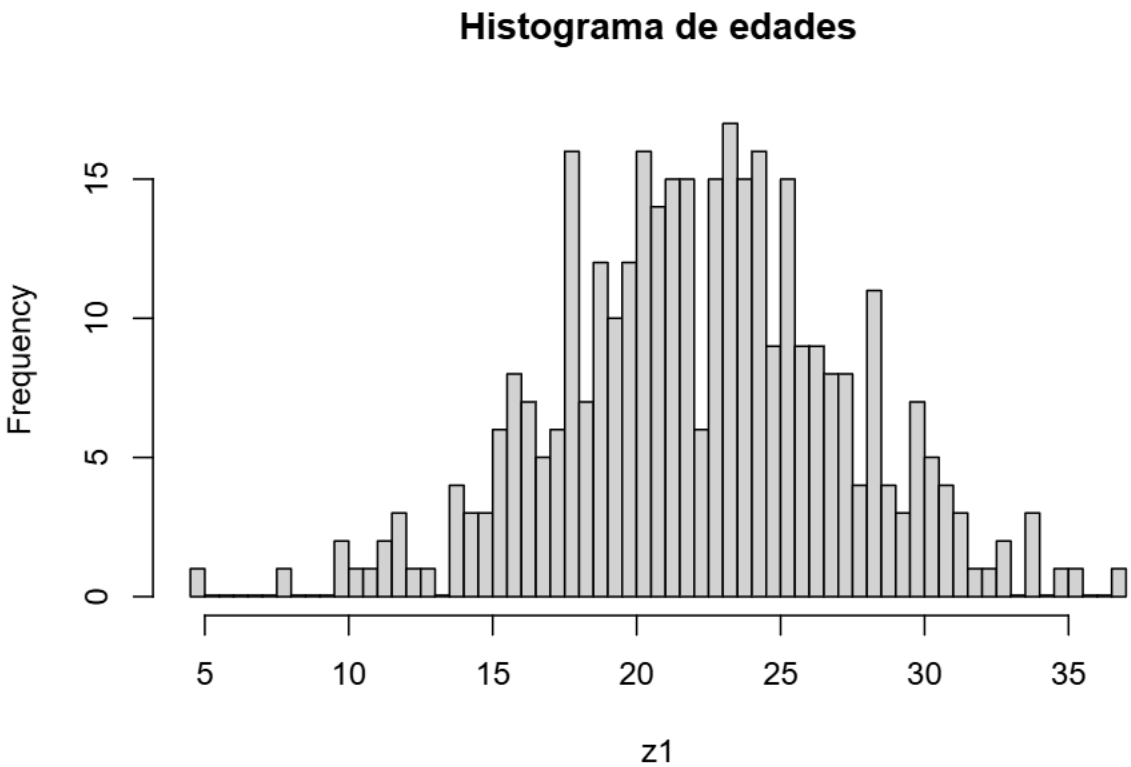
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



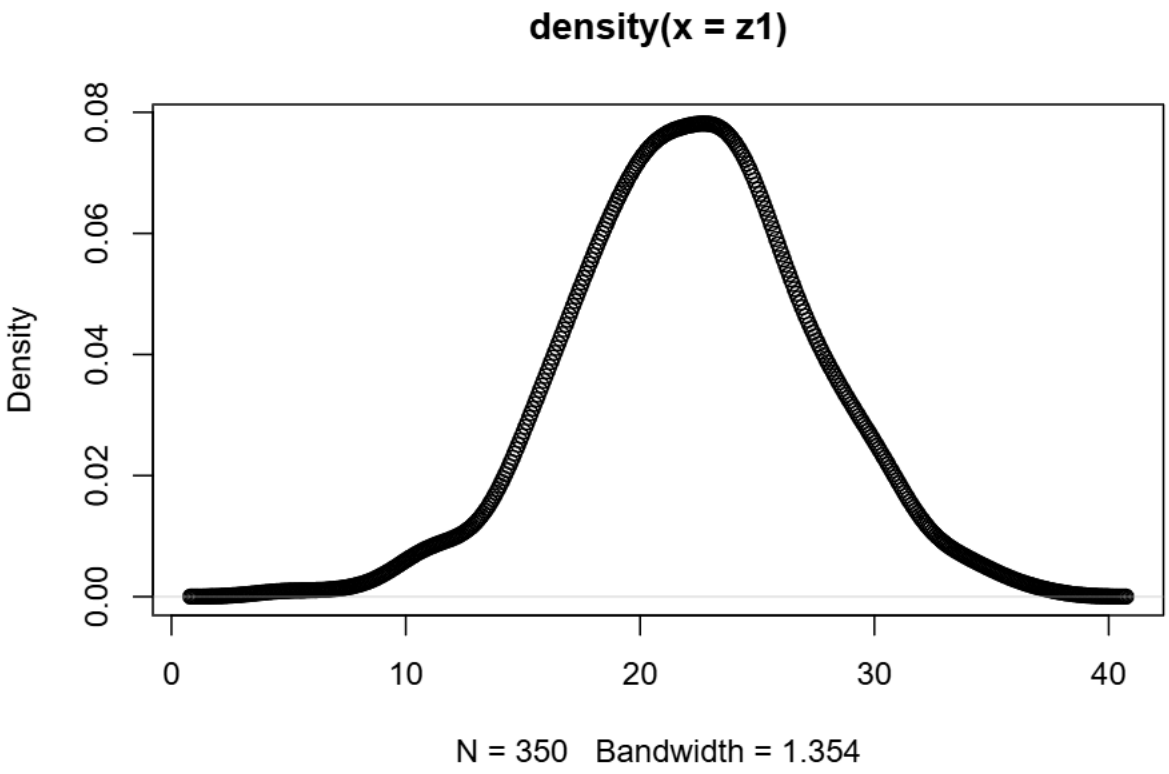
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.354
##
##        x                 y
##  Min.   : 0.8077   Min.   :9.540e-06
##  1st Qu.:10.7912   1st Qu.:1.380e-03
##  Median :20.7746   Median :1.064e-02
##  Mean   :20.7746   Mean   :2.502e-02
##  3rd Qu.:30.7580   3rd Qu.:4.663e-02
##  Max.   :40.7414   Max.   :7.815e-02
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
```

```
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
```

```python
secuencia_dni = list(range(1, dni + 1))

# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.202743 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.
```

```r
#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.005 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.006810427 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.002198696 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
```

```
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
```

```r
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  20.446   0.016  20.569
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.017   0.000   0.016
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Comparación:

```r
suma1
```

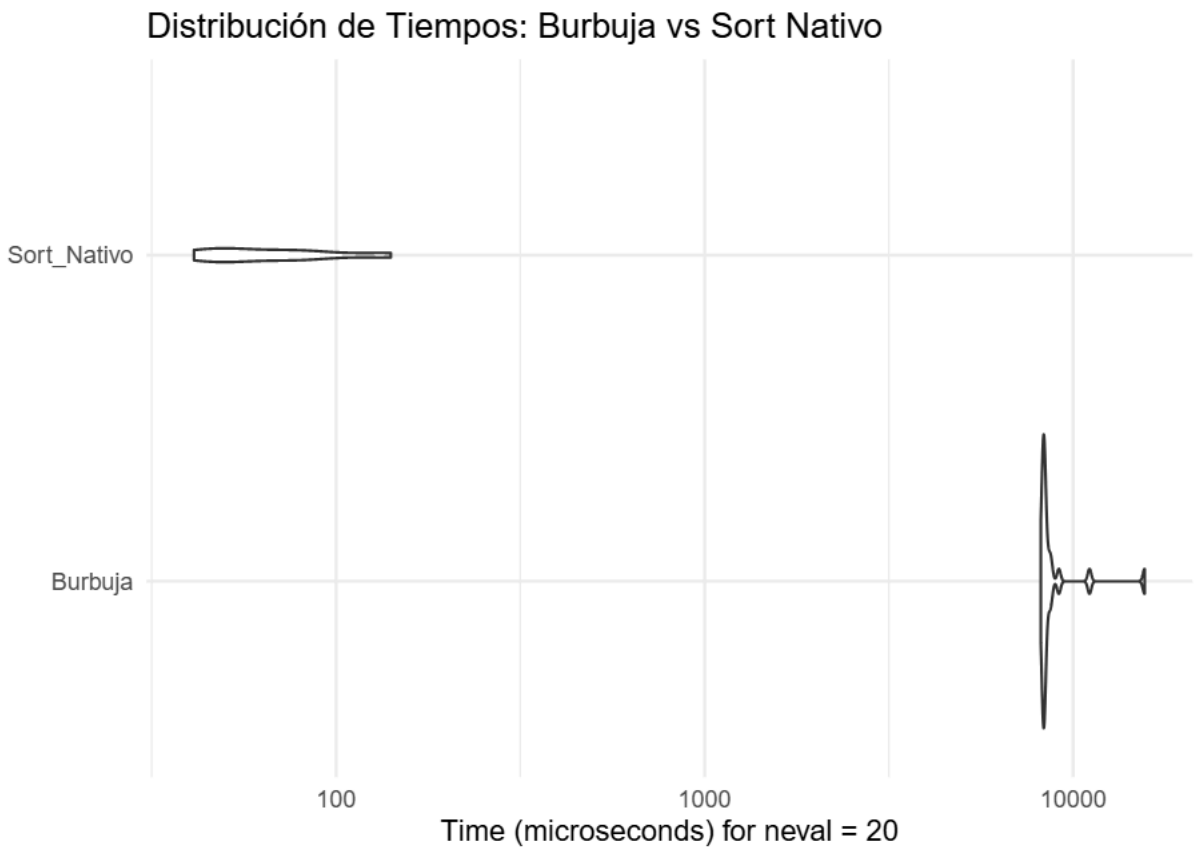
```
## [1] 500000500000
```

```r
suma2
```

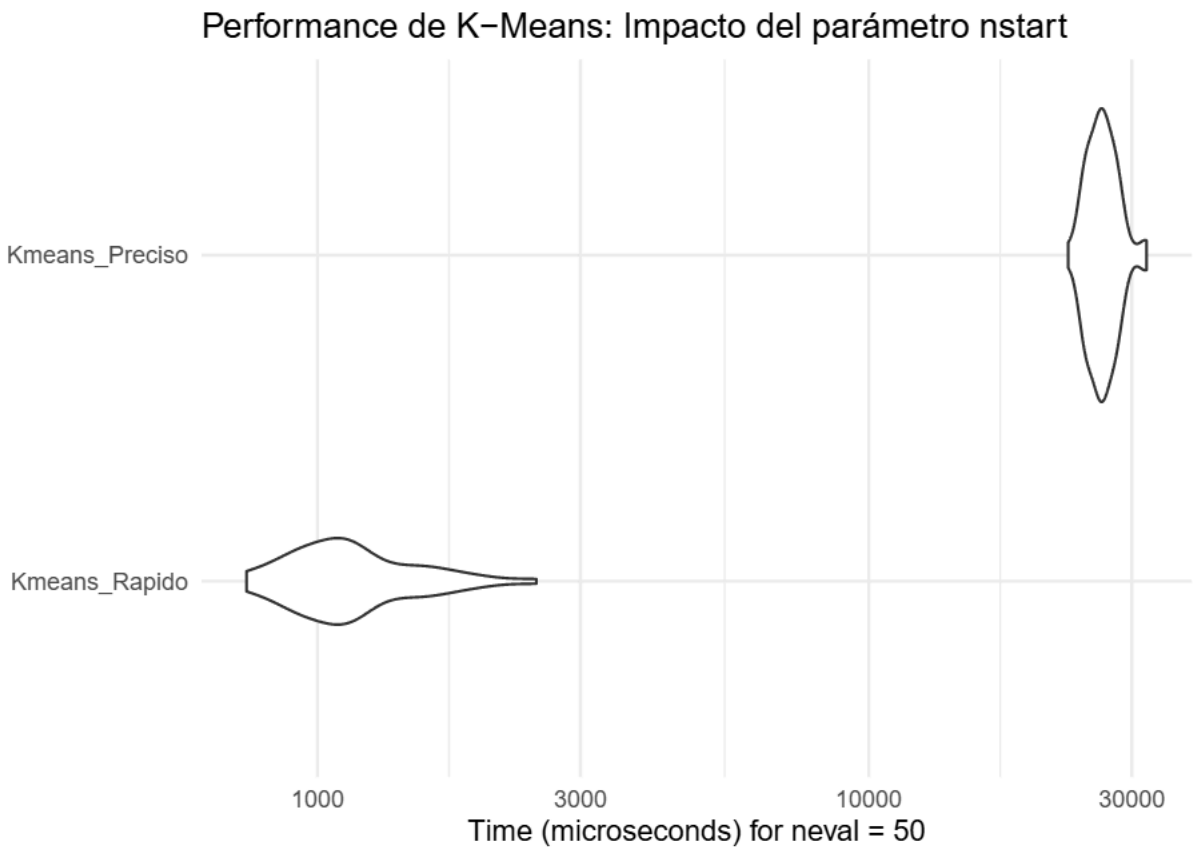
```
## [1] 500000500000
```

Análisis de Rendimiento: Ordenamiento y Agrupación

```r
library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(
  Burbuja = burbuja(muestra),
  Sort_Nativo = sort(muestra),
  times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()
```




```r
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()
```



Conclusión final

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos empíricamente que la calidad de un código no radica solo en llegar al resultado correcto, sino en cómo optimiza el tiempo y los recursos del sistema.

La incorporación de los gráficos de violín resultó fundamental para visualizar estadísticamente no solo la media de ejecución, sino la estabilidad y distribución temporal de procesos analíticos complejos, como la comparación de métodos de ordenamiento y la medición de performance del agrupamiento K-Means. Es fundamental destacar que los tiempos absolutos medidos y la forma exacta de la "panza" de los violines serán diferentes en cada computadora; estos resultados variarán dinámicamente según las propiedades físicas y la capacidad de procesamiento (hardware) del equipo donde se compile el documento.

Sin embargo, más allá de la máquina específica que se utilice, el principio de optimización estructural se mantiene inalterable. El ejercicio de "La penitencia de Newton" ilustró perfectamente este concepto: mientras la fuerza bruta de un bucle iterativo demanda un costo computacional inmenso para realizar operaciones masivas, la aplicación de un modelo matemático sólido resuelve el problema en fracciones de segundo.

En definitiva, la evaluación de estos algoritmos nos confirma que la herramienta de optimización más poderosa en la programación moderna no es la computadora que tengamos, sino el razonamiento lógico, físico y matemático que aplicamos antes de escribir la primera línea de código.